CYBERSECURITY & APPLICATION LOGIC AUDIT

UniConv Platform Security Audit

Adversarial Code Review, OWASP Top 10 Vulnerability Scan & Defensive Quality Assessment

Target: uniconv-main | Scope: FastAPI & Next.js | Classification: RESTRICTED

1. Executive Summary & Assessment Scope

During an adversarial security and logic audit of the UniConv web application (encompassing both the FastAPI document processing backend and Next.js frontend), an elite application security assessment was conducted. The objective was to identify OWASP Top 10 vulnerabilities, broken authorization flows, injection risks, unhandled logic edge-cases, and insecure cryptographic configurations.

The assessment identified multiple critical access control flaws that enable complete tenant impersonation, client-side administrative bypasses, unauthenticated email spoofing, and severe denial-of-service/SSRF attack vectors. The overall security posture is evaluated as HIGH RISK. Immediate remediation is mandatory prior to production deployment.

Vulnerability Severity Breakdown

CRITICAL	HIGH	MEDIUM	LOW / QUALITY
4 Findings	5 Findings	3 Findings	1 Finding

Key Architectural Risk Areas

* Identity & Access Management: Complete absence of backend JWT validation allows arbitrary user_id impersonation. Admin route protection relies purely on client-side React useEffect checks with publicly exposed admin emails.
* Media & Conversion Pipeline: FFmpeg and LibreOffice process untrusted inputs without timeouts or protocol restrictions, exposing the service to SSRF, local file disclosure, and permanent CPU worker lockups.
* Data Transport & Session Security: Permissive CORS with credential reflection compromises session tokens; unauthenticated mail endpoints permit arbitrary spam relaying; response splitting risks exist on file download headers.

2. Audit Findings Matrix

The following table indexes all security vulnerabilities and logic defects identified during the audit:

ID	Severity	Category	Target File / Function	Vulnerability Summary
SEC-01	CRITICAL	Broken Access	main.py: create_job()	IDOR & Auth Bypass in Job Queue
SEC-02	CRITICAL	Broken Access	admin/page.tsx	Client-Side Admin Check & PII Leak
SEC-03	CRITICAL	Auth Failure	middleware.ts	Insecure getSession() in Route Guard
SEC-04	CRITICAL	Relay / XSS	email_service.py	Unauthenticated Mail Relay & Phishing
SEC-05	HIGH	SSRF / DoS	media_service.py	FFmpeg SSRF & Hanging Subprocesses
SEC-06	HIGH	Injection	convert/route.ts	Header Injection (CRLF) & Buffer DoS
SEC-07	HIGH	Misconfig	main.py: CORS	allow_origins=['*'] with Credentials
SEC-08	HIGH	Crypto	pdf_service.py	Hardcoded 'admin' Password for PDF
SEC-09	HIGH	Supply Chain	MainWorkspace.tsx	Unsanitized 3rd-Party Ad DOM Scripts
BUG-10	MEDIUM	Path Traversal	main.py: worker	Unsanitized Local File Path Staging
BUG-11	MEDIUM	Logic Defect	main.py: webhook	Database Schema Mismatch Breaks Upgrades
BUG-12	MEDIUM	Reliability	pdf_service.py	FD Leaks & Excel Sheet Title Crashes
SEC-13	LOW	Hardening	Dockerfile	Container Execution as Root User

3. In-Depth Vulnerability Technical Dossiers

Detailed root cause analysis, exploit mechanics, impact assessment, and drop-in code fixes:

[SEC-01] Complete Broken Access Control & IDOR in Job Creation

SEVERITY	CATEGORY	TARGET	CWE ID
CRITICAL (CVSS 9.8)	OWASP A01: Broken Access	apps/api/src/main.py:184-262	CWE-284 / CWE-639

Vulnerability Description:
The /api/jobs endpoint accepts user_id as an untrusted query parameter without requiring or verifying an HTTP Authorization Bearer JWT token. Any caller can provide an arbitrary UUID to impersonate victims, exhaust account quotas, or manipulate jobs. Furthermore, callers can specify arbitrary file_id values belonging to other users (Insecure Direct Object Reference). Additionally, for anonymous users where user_id is omitted, line 250 skips setting job_data['user_id'], which subsequently triggers an unhandled KeyError: 'user_id' in process_document_job (line 453), unconditionally crashing all guest conversion tasks.

PROOF / ATTACK SCENARIO

Attack Scenario:
An adversary sends: POST /api/jobs?file_id=<target_file_uuid>&user_id=<admin_uuid>
The backend accepts the unauthenticated request, validates tier limits against the admin's unlimited quota, processes the victim's file, and saves the output to the admin's files record without any cryptographic verification.

Remediation / Secure Drop-In Replacement:

```
# apps/api/src/main.py
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from fastapi import Security, Depends, HTTPException, BackgroundTasks

security = HTTPBearer(auto_error=False)

async def get_current_user_optional(creds: Optional[HTTPAuthorizationCredentials] = Security
    (security)):
    if not creds or not supabase:
        return None
    try:
        res = supabase.auth.get_user(creds.credentials)
        if res and res.user:
            return {"id": res.user.id, "email": res.user.email}
    except Exception:
        pass
    return None

@app.post("/api/jobs")
async def create_job(request: JobRequest, file_id: str, background_tasks: BackgroundTasks,
                     current_user: Optional[dict] = Depends(get_current_user_optional)):
    user_id = current_user["id"] if current_user else None

    file_res = supabase.table("files").select("id, size_bytes, user_id").eq("id", file_id).e
    xecute()
    if not file_res.data:
        raise HTTPException(status_code=404, detail="File not found")
    target_file = file_res.data[0]
    if target_file.get("user_id") and target_file["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="Access to file denied")

    job_data = {
        "tool": request.tool, "input_file_ids": request.input_file_ids or [file_id],
        "configuration": request.configuration or {}, "status": "QUEUED",
        "user_id": user_id  # Explicit None avoids KeyError in worker
    }
    job = supabase.table("processing_jobs").insert(job_data).execute().data[0]
    background_tasks.add_task(process_document_job, job["id"])
    return {"job_id": job["id"], "status": "QUEUED"}
```

[SEC-02] Client-Side Admin Authorization & Direct Database Manipulation

SEVERITY	CATEGORY	TARGET	CWE ID
CRITICAL (CVSS 9.1)	OWASP A01: Broken Access	apps/web/src/app/admin/page.tsx:23-109	CWE-602 / CWE-269

Vulnerability Description:
Administrative authorization in /admin is implemented exclusively in client-side browser JavaScript via useEffect. The fallback admin email ('4mh24cs167@gmail.com') is embedded into client bundles and rendered in login placeholders. Furthermore, the admin view performs direct Supabase queries with NEXT_PUBLIC_SUPABASE_ANON_KEY to read all registered users and update plan tiers. Because public RLS policies permit anonymous reads and writes (as noted in upload.ts:8-9), any authenticated or guest user can bypass the UI check to dump all customer PII and elevate accounts to Premium.

PROOF / ATTACK SCENARIO

Attack Scenario:
A regular user registers an account, opens the browser console, and executes:
const { data } = await supabase.from('plans').select('id').eq('name', 'Premium').single();
await supabase.from('users').update({ plan_id: data.id }).eq('id', (await supabase.auth.getUser()).data.user.id);
The database immediately grants the user Premium privileges without administrative consent or payment.

Remediation / Secure Drop-In Replacement:

```
// apps/web/src/app/api/admin/upgrade-user/route.ts
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@supabase/supabase-js";

// Private service role client - never exposed to browser
const supabaseAdmin = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.SUPABA
    SE_SERVICE_ROLE_KEY!);

export async function POST(req: NextRequest) {
  const token = req.headers.get("Authorization")?.replace("Bearer ", "");
  if (!token) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

  const { data: { user }, error } = await supabaseAdmin.auth.getUser(token);
  // Verify against unexposed server environment secret
  if (error || !user || user.email !== process.env.ADMIN_EMAIL) {
    return NextResponse.json({ error: "Forbidden: Admin required" }, { status: 403 });
  }

  const { targetUserId, newPlanName } = await req.json();
  const { data: plan } = await supabaseAdmin.from("plans").select("id").eq("name", newPlanNa
    me).single();
  if (!plan) return NextResponse.json({ error: "Invalid plan" }, { status: 400 });

  await supabaseAdmin.from("users").update({ plan_id: plan.id }).eq("id", targetUserId);
  return NextResponse.json({ success: true });
}
```

[SEC-03] Insecure getSession() in Server-Side Authentication Middleware

SEVERITY	CATEGORY	TARGET	CWE ID
CRITICAL (CVSS 9.1)	OWASP A07: Identification Failures	apps/web/src/middleware.ts:57-74	CWE-287 / CWE-345

Vulnerability Description:
The Next.js edge middleware guards protected routes using supabase.auth.getSession(). Per official Supabase security guidance, getSession() reads raw cookie structures without validating token signatures or revocation status against the Supabase Auth server. Attackers can present forged or tampered session tokens to evade middleware redirects. Moreover, the middleware matcher completely omits /admin, leaving administrative pages unprotected at the edge.

Remediation / Secure Drop-In Replacement:

```
// apps/web/src/middleware.ts
export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request: { headers: request.headers } });
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { /* standard getter/setter */ } }
  );

  // SECURE: getUser() cryptographically validates the JWT against Supabase Auth
  const { data: { user }, error } = await supabase.auth.getUser();
  const isAuth = !!user && !error;
  const path = request.nextUrl.pathname;

  if ((path.startsWith('/dashboard') || path.startsWith('/admin')) && !isAuth) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
  return response;
}
export const config = { matcher: ['/dashboard/:path*', '/admin/:path*', '/login', '/register
    '] };
```

[SEC-04] Unauthenticated Email Relay & HTML Injection / Phishing

SEVERITY	CATEGORY	TARGET	CWE ID
CRITICAL (CVSS 9.3)	OWASP A01 / A03: Injection	apps/api/src/main.py:482 / email_service	CWE-306 / CWE-79

Vulnerability Description:
The /api/admin/notify-upgrade endpoint is completely unauthenticated and accessible to any external caller. In email_service.py, the user-supplied plan_name is concatenated raw into both the HTML email body and subject line. Threat actors can utilize this endpoint as an open spam relay to dispatch arbitrary emails from the company's verified Gmail SMTP address, or inject malicious HTML markup, phishing credentials forms, and phishing links.

PROOF / ATTACK SCENARIO

Attack Scenario:
curl -X POST https://uniconv.onrender.com/api/admin/notify-upgrade -H 'Content-Type: application/json' \
-d '{"user_email":"victim@company.com","plan_name":"Pro</h1>Reset Credentials<!--"}'
The victim receives an official email from the legitimate UniConv domain containing active credential phishing links.

Remediation / Secure Drop-In Replacement:

```
# apps/api/src/services/email_service.py
import html, smtplib, re, os
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText

class EmailService:
    @staticmethod
    def send_upgrade_email(user_email: str, plan_name: str) -> bool:
        sender_email, sender_password = os.getenv("SMTP_EMAIL"), os.getenv("SMTP_PASSWORD")
        if not sender_email or not sender_password or not re.match(r"^[\w\.-]+@[\w\.-]+\.\w+
    $", user_email):
            return False

        # Whitelist permitted values & HTML escape
        safe_plan = "Pro" if "pro" in plan_name.lower() else "Premium"
        body = f"<html><body><h2>Account Upgraded</h2><p>Plan: <strong>{html.escape(safe_pla
    n)}</strong></p></body></html>"

        msg = MIMEMultipart("alternative")
        msg["From"], msg["To"], msg["Subject"] = f"UniConv <{sender_email}>", user_email, f"
    UniConv {safe_plan} Upgrade"
        msg.attach(MIMEText(body, "html", "utf-8"))

        try:
            with smtplib.SMTP("smtp.gmail.com", 587, timeout=10.0) as server:
                server.starttls()
                server.login(sender_email, sender_password)
                server.send_message(msg)
            return True
        except Exception as e:
            return False
```

[SEC-05] SSRF & Local File Inclusion / Resource Starvation via FFmpeg

SEVERITY	CATEGORY	TARGET	CWE ID
HIGH (CVSS 8.8)	OWASP A10: SSRF & A05: Misconfig	apps/api/src/services/media_service.py	CWE-918 / CWE-400

Vulnerability Description:
1. FFmpeg is executed without disabling network and file protocol access (-protocol_whitelist 'file'). An attacker can supply a crafted HLS playlist (.m3u8) or AVI container referencing internal metadata (http://169.254.169.254) or container paths (file:///app/.env) to exfiltrate database keys and cloud secrets.
2. subprocess.run() is executed without timeouts. Malicious decompression bombs or complex Office formulas cause FFmpeg and LibreOffice processes to hang indefinitely, depleting all backend worker threads (Denial of Service).

Remediation / Secure Drop-In Replacement:

```
# apps/api/src/services/media_service.py
class MediaService:
    DEFAULT_TIMEOUT = 120 # 2 minute hard timeout per subprocess

    @staticmethod
    def extract_audio(input_path: str, output_path: str) -> bool:
        try:
            cmd = ["ffmpeg", "-nostdin", "-y", "-protocol_whitelist", "file", "-i", input_pa
    th, "-q:a", "0", "-map", "a", output_path]
            subprocess.run(cmd, check=True, timeout=MediaService.DEFAULT_TIMEOUT, stdout=sub
    process.DEVNULL, stderr=subprocess.DEVNULL)
            return True
        except (subprocess.TimeoutExpired, subprocess.CalledProcessError):
            return False
```

[SEC-06] HTTP Response Header Injection (CRLF) & Buffer DoS

SEVERITY	CATEGORY	TARGET	CWE ID
HIGH (CVSS 8.2)	OWASP A03: Injection & A05: Misconfig	apps/web/src/app/api/convert/route.ts	CWE-113 / CWE-400

Vulnerability Description:
In /api/convert/route.ts, format is interpolated directly into the Content-Disposition header. CRLF characters (\r\n) allow attackers to inject malicious headers (Set-Cookie) or trigger HTTP Response Splitting. In remove-watermark/route.ts, file.type and file.name are reflected verbatim into response headers, enabling Reflected XSS. Furthermore, calling await file.arrayBuffer() directly loads entire multi-gigabyte uploads into Node.js heap memory, crashing the server.

Remediation / Secure Drop-In Replacement:

```
// apps/web/src/app/api/convert/route.ts
const ALLOWED_FORMATS = new Set(["png", "jpg", "jpeg", "webp", "pdf", "xlsx", "csv", "mp3",
    "mp4"]);
const MAX_FILE_SIZE = 25 * 1024 * 1024; // 25MB buffer cap

export async function POST(req: NextRequest) {
  const formData = await req.formData();
  const file = formData.get("file") as File | null;
  const format = (formData.get("format") as string || "").toLowerCase().trim();

  if (!file || !ALLOWED_FORMATS.has(format)) return NextResponse.json({ error: "Invalid inpu
    t" }, { status: 400 });
  if (file.size > MAX_FILE_SIZE) return NextResponse.json({ error: "File exceeds 25MB limit"
     }, { status: 413 });

  const buffer = Buffer.from(await file.arrayBuffer());
  const outputBuffer = await sharp(buffer).toFormat(format as any).toBuffer();

  return new NextResponse(outputBuffer, {
    headers: {
      "Content-Type": `image/${format}`,
      "Content-Disposition": `attachment; filename="converted.${encodeURIComponent(format)}"
    `,
      "X-Content-Type-Options": "nosniff"
    }
  });
}
```

[SEC-07] Permissive CORS with Dynamic Credential Reflection

SEVERITY	CATEGORY	TARGET	CWE ID
HIGH (CVSS 8.1)	OWASP A05: Security Misconfig	apps/api/src/main.py:13-19	CWE-942

Vulnerability Description:
FastAPI is configured with allow_origins=['*'] and allow_credentials=True. In Starlette/FastAPI, this combination dynamically reflects the requesting client's Origin header in Access-Control-Allow-Origin with credentials: true. Any external malicious website visited by an authenticated user can perform authenticated background requests to read API data.

Remediation / Secure Drop-In Replacement:

```
# apps/api/src/main.py
app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv("FRONTEND_URL", "https://uniconv.vercel.app").rstrip("/"), "htt
    p://localhost:3000"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type"],
)
```

[SEC-08] Hardcoded Static Master Password Defeating PDF Restrictions

SEVERITY	CATEGORY	TARGET	CWE ID
HIGH (CVSS 7.5)	OWASP A02: Cryptographic Failures	apps/api/src/services/pdf_service.py:327	CWE-798 / CWE-321

Vulnerability Description:
In PDFService.secure_permissions, owner_pw='admin' is hardcoded when generating encrypted PDFs. Anyone who receives a document restricted against printing or editing can supply 'admin' to bypass all security flags immediately.

```
# apps/api/src/services/pdf_service.py
import secrets
owner_pw = owner_password if owner_password else secrets.token_urlsafe(32)
doc.save(output_path, encryption=fitz.PDF_ENCRYPT_AES_256, permissions=perms, owner_pw=owner
    _pw)
```

[SEC-09] Third-Party Script Supply-Chain & Malvertising DOM Hijacking

SEVERITY	CATEGORY	TARGET	CWE ID
HIGH (CVSS 7.9)	OWASP A08: Software Integrity Failures	apps/web/src/components/MainWorkspace.tsx	CWE-829 / CWE-79

Vulnerability Description:
The application dynamically appends external scripts from third-party ad networks (highrevenueformat.com, profitableratecpmnetwork.com) directly into the main DOM. These unvetted scripts run with full access to localStorage, session tokens, and uploaded user files, creating an acute supply chain and malvertising compromise risk.

```
// Secure isolated iframe wrapper for external third-party advertisements
export const SandboxedAd = ({ adKey, w, h }: { adKey: string; w: number; h: number }) => (
  <iframe
    srcDoc={`<!DOCTYPE html><html><body><script src="https://www.highrevenueformat.com/${adK
    ey}/invoke.js"></script></body></html>`}
    width={w} height={h} sandbox="allow-scripts allow-popups" className="border-0 overflow-h
    idden" title="Ad"
  />
);
```

[BUG-10 to SEC-13] Medium & Low Severity Flaws / Logic Defects

BUG-10: Path Traversal Vulnerability in Worker File Staging (apps/api/src/main.py:299)

file_metadata['filename'] is taken directly from user upload strings. Using os.path.join(temp_dir, f'{f_id}_{filename}') allows path escape sequences. Fix: Sanitize filename with os.path.basename and whitelist characters [a-zA-Z0-9._-].

BUG-11: Database Schema Mismatch Breaking Webhook Upgrades (apps/api/src/main.py:164)

The webhook writes to column 'plan' instead of foreign key 'plan_id', raising a database schema error on every paid upgrade. Fix: Resolve plans.id from the plans table using the plan name before updating users.plan_id.

BUG-12: File Descriptor Leaks & Excel Sheet Character Crashes (image_service.py / pdf_service.py)

Image.open() instances in jpg_to_pdf are never closed, leaking file descriptors. In pdf_to_excel, special characters in PDF table headers crash openpyxl sheet creation. Fix: Use with Image.open() blocks and sanitize sheet names.

SEC-13: Docker Container Running as Root (apps/api/Dockerfile:1-30)

The container executes Uvicorn, FFmpeg, and LibreOffice as root. Exploit payloads targeting document parsers yield root container execution. Fix: Add RUN useradd -r appuser && USER appuser.

4. Strategic Remediation Roadmap

To transition the UniConv platform to an enterprise-grade security posture, follow this 3-phase roadmap:

Phase 1: Immediate Containment (Days 1 - 3)

1. Deploy JWT authentication verification to all FastAPI routes (/api/jobs, /api/subscriptions).
2. Implement Postgres Row-Level Security (RLS) on files and processing_jobs: auth.uid() = user_id.
3. Secure the admin portal with server-side Server Actions and private service-role keys; eliminate client-side checks.
4. Enforce strict origin filtering in CORSMiddleware.

Phase 2: Processing & Pipeline Hardening (Days 4 - 7)

1. Sandbox FFmpeg and LibreOffice with 60-second timeouts, -protocol_whitelist 'file', and memory cgroups.
2. Replace direct file.arrayBuffer() reads with size-gated streams and input validation.
3. Quarantine third-party ad scripts into sandboxed, origin-restricted iframes.
4. Fix webhook database schema queries to accurately update plan_id.

Phase 3: Production Infrastructure & Compliance (Weeks 2 - 3)

1. Migrate background processing from FastAPI BackgroundTasks to an asynchronous Redis + Celery worker cluster.
2. Convert public storage buckets into private buckets, issuing 5-minute single-use signed download URLs.
3. Rebuild API Docker container to execute under an unprivileged appuser with no-new-privileges flags.
4. Establish automated CI/CD static security scanning (Bandit for Python, Semgrep, and npm audit).

5. Auditor Sign-Off

AUDIT LEAD	METHODOLOGY	STATUS	REPORT VERSION
Antigravity AppSec Team	**OWASP ASVS / Top 10**	**Audit Completed**	**v1.0 (Final)**

CONFIDENTIAL -- FOR INTERNAL SECURITY AND REMEDIATION USE ONLY.